PRODUCT REQUIREMENTS DOCUMENT / RETROSPECTIVE

Photo Viewer

A small product, deliberately designed.

A native macOS viewer for opening a photo, moving through its folder and inspecting detail - without editing the original or managing another photo library.

Product owner: Nikhil Sharma

Nikhil defined the problem, set the constraints, reviewed real usage, refined the interaction design and directed trade-offs. Codex, OpenAI's AI coding assistant, assisted with implementation, testing, investigation and documentation.

The product thesis

A focused utility can deliver value through restraint. Keep the photo central, make navigation predictable, reveal just enough file context, and preserve trust through read-only behaviour.

What this case study demonstrates

- **Scope discipline:** one reusable image window, no editing tools and no library-management workflow.

- **Iterative product judgement:** direct feedback on keyboard navigation, layout, zoom clarity and metadata became testable requirements.

- **Evidence-led optimisation:** improve the actual Swift/ImageIO bottlenecks before considering a language rewrite.

Snapshot and document status

11 October 2026. Implemented personal-use macOS application; public open-source release is proposed, not published. Target: macOS 26 or later on Apple silicon. Latest local evidence uses Xcode 27 / Swift 6.4 on macOS 27.0.1.

This is a retrospective PRD reconstructed from the request history and current source. It is not a claim that a formal PRD, research programme or roadmap existed before development. Iteration dates are not invented.

01 / PROBLEM AND PRODUCT STRATEGY

Make viewing feel effortless

Primary user: a MacBook owner who opens images from Finder and wants to inspect neighbouring photos quickly.

Problem statement

The desired workflow is viewing, not editing or importing. Extra windows, unreliable arrow navigation, ambiguous zoom indicators and scattered metadata interrupt a simple task: inspect this image, then the next one.

Jobs to be done

- When I open an image from Finder, show it large enough to inspect immediately, without taking over the entire display.

- When a folder mixes common image formats, let me browse it with the same two keys and a predictable sequence.

- When I zoom or compare files, tell me where I am, what scale I am viewing and which file is on screen.

Goal	Product response	Evidence to seek
Reduce interaction cost	Finder/drop entry; default Fit; one image window	Open, navigate and recover without extra controls
Make state understandable	Active Fit treatment; moving percentage; photo position	User can distinguish Fit from manual zoom
Protect originals	No editing commands; read-only image access	Content and metadata invariance checks
Stay responsive and lightweight	Background indexing, display-sized decode, bounded RAM cache	Stage timings, queue bounds and memory checks

Explicit non-goals

Editing, renaming, deletion, annotations, thumbnails, slideshows, RAW support, cloud-library integration, accounts, analytics and synchronisation. No iOS app is in scope. App Store distribution, Intel support and notarised public binaries were not part of the personal-use delivery.

Research boundary

The evidence is one product owner's expressed needs and hands-on feedback. No interviews, market sizing, adoption data or statistically validated usability study are claimed.

02 / THE PM JOURNEY

From intent to precise behaviour

The sequence below captures the decisions, not invented dates or a reconstructed sprint schedule.

Feedback / signal	Product decision	Why it mattered
A minimalist personal viewer	One resizable, large window; Finder entry; no editing	Set a narrow job and explicit boundaries before adding features
Two Open With entries; arrows failed on Desktop images	Treat installation registration and keyboard/folder access as core reliability work	The basic browse journey must work before refinement
How is the order decided? Need creation date and size	Natural filename order; explicit file metadata and Photo x of y	Make sequence and file identity visible
Zoom label wrapped and looked attached to the + button	Remove redundant Fit label and text field; place small percentage above the slider thumb	Reduce ambiguity rather than add explanatory clutter
Is Fit on? Are min/max captions aligned?	Checked, filled Fit state; fixed caption/icon geometry; 20% and 200% labels	Turn a visual preference into an explicit state contract
Could Rust make it faster? Will caches grow?	Retain native Swift; remove repeated work; downsample Fit; bound RAM-only cache	Optimise measured costs while limiting complexity and storage risk
Credit the creator and AI; share the process	Menu-based About; build date; retrospective product artifacts; publication review	Make ownership and AI assistance transparent

How the PM role shows up

Nikhil moved from a broad experience goal to precise interaction rules, used real defects as prioritisation signals, challenged unclear controls, and asked for both performance evidence and operational consequences. The implementation work supported these decisions; it did not replace product ownership.

Retired requirements remain part of the story: the editable percentage field and a separate right-side Fit/percentage status were explicitly superseded. They are not current acceptance criteria.

03 / FUNCTIONAL SCOPE

The current product contract

Requirement IDs align with the epics and stories in the companion delivery backlog.

ID / priority	Requirement	Boundary
R1 / P0	Finder open events and image drops route to one reusable image window	A new external open replaces context and resets Fit; no File Open button
R2 / P0	Normal macOS window controls, about 90% initial usable display, restored frame afterward	Resizable, minimum 640 x 480 points; green control zooms, not fullscreen
R3 / P0	Left/Right browses supported siblings in natural filename order	No wrapping; keep zoom mode; folder permission failures have a recoverable notice
R4 / P0	Common native image formats work together in a folder	JPEG, PNG, HEIC/HEIF, TIFF, GIF, BMP and WebP; additional native formats recognised
R5 / P0	Fit plus 20-200% manual zoom, in 5% steps; mouse/trackpad panning	Fit is not constrained by manual bounds; 100% maps image pixels to screen pixels
R6 / P1	Two-row file context plus unambiguous compact zoom controls	No editable metadata, standalone zoom text box or visible app version
R7 / P0	Read-only image handling and recoverable decode errors	No save, rename, delete, rotation or metadata writes; first animation frame only
R8 / P1	Responsive loading and bounded temporary storage	Async work, latest-request scheduling, 128 MiB RAM cache; not a total RAM cap
R9 / P1	Menu-based About and buildable local delivery	Creator, AI assistance and build date; one separate reusable About panel
R10 / proposed	Prepare a safe and reproducible open-source release	Requires license, clean staging, documentation and release checks before publishing

P0 = essential viewing and trust contract. P1 = important clarity, efficiency or delivery quality. Priorities are retrospective groupings, not a claim about the original implementation order.

04 / INTERACTION DESIGN

State should explain itself

Header information architecture

Two compact lines

Line 1: filename · full file path
Line 2: Created date: value · Size: value · Photo x of y

Creation date means filesystem creation date, not EXIF capture time. Size means the logical file size. Missing values are shown as unavailable, never fabricated. Long text may truncate; help text and accessibility labels retain the full context.

Zoom state model

Interaction	Result
External Finder/drop open	Set Fit and calculate the effective display percentage
Left/Right to another image	Preserve Fit or the chosen manual percentage
Move slider or press - / +	Enter manual mode; snap/clamp to 20-200% in 5% steps
Click Fit	Return to automatic sizing; checked/filled Fit indicates it is on
Resize / change screen scale	Recompute Fit; keep 100% equal to one image pixel per physical screen pixel

The small percentage sits above the thumb and shares a caption baseline with Min 20% and Max 200%. In Fit, the label shows actual scale; the thumb uses the nearest supported manual position and clamps at the ends. For example, Fit at 12% can show 12% while the thumb rests at 20%.

Browse flow

OPEN Finder or drop → **VIEW** Fit + file context → **NAVIGATE** Left / Right → **INSPECT** Zoom and pan

Natural order is filename-based, not creation-date order: photo2.jpg comes before photo10.png. Only supported file candidates in the immediate parent folder participate; no recursive folder browsing or wrapping.

Exceptions do not become dead ends

A corrupt or missing image shows a quiet error in the image area. The active folder list and arrow navigation remain usable. Folder-access failures explain the navigation limitation and can be retried. An active text editor, when present, retains its arrow keys.

05 / TECHNICAL AND OPERATIONAL DESIGN

Optimise the work, not the language

SwiftUI provides the compact interface; AppKit provides precise window, keyboard and scrolling behaviour. Apple ImageIO handles native decoding.



Design choice	Rationale	Trade-off / limit
Background folder snapshot	Scan, natural-sort and index once; arrows reuse the result	Initial discovery still costs time; folder changes need refresh
Display-sized Fit decode	Avoid a full-resolution pixel buffer just to draw a smaller image	Manual zoom upgrades to original detail; preview remains while loading
One active + latest pending decode	Rapid browsing cannot accumulate a decode backlog	An already-running synchronous ImageIO decode finishes; obsolete results are ignored
128 MiB decoded-byte LRU cache	Speed up revisits without accumulating photo copies on disk	Current image and decoder allocations sit outside the cache budget
No speculative neighbour prefetch	Avoid surprise reads/downloads from cloud-backed folders	An unseen next photo may still require decoding

File change and display correctness

Folder notifications are debounced, with boundary retries and a lightweight foreground fallback. Cached image entries validate file identity, size and modification time. Original orientation-correct dimensions remain separate from preview dimensions, preserving Fit percentages and pixel-accurate manual scaling.

Storage promise

The app creates no persistent image cache. Cache contents live in RAM and disappear when the process quits. Closing the window alone leaves the macOS app running. Window preferences may persist normally; this is not a promise of zero operating-system storage activity.

Why not Rust?

The identified costs were repeated folder work and unnecessary full-resolution decoding, not a custom CPU-heavy algorithm. Improving those paths in Swift kept the native stack simple. A rewrite had no measured benefit to justify it.

06 / EVIDENCE AND SUCCESS MEASURES

Measured gains, bounded claims

The results below are local engineering evidence, not a whole-app performance score or a claim about every image.

Synthetic 48 MP JPEG	Before / full detail	After / Fit-sized	Change
Median ImageIO decode	385.16 ms	273.39 ms	29.0% less time
Decoded dimensions	8000 x 6000	2560 x 1920	Display-sized
Decoded pixel buffer	192.0 MB	19.66 MB	89.8% smaller

7 timed samples per decode size; warm-cache generated JPEG; Swift 6.4 optimised build on macOS 27.0.1. Pixel-buffer bytes are not peak process RAM. These are stage-specific comparisons, not launch time or keypress-to-display timings.

Navigation bookkeeping

For synthetic folders of 100 / 1,000 / 5,000 image candidates, repeated scan/sort plus last-item lookup measured 19.89 / 205.83 / 1,013.51 ms. Cached lookup was below 0.001 ms per operation. Initial index construction, decoding and rendering are excluded; do not present this as an equivalent overall app speedup.

Verification available at this snapshot

Evidence	What it supports	What it does not establish
53 tests; 0 failures	State 19, catalog 8, loader 10, window 10, zoom 6; latest saved run checked	No guarantee of every UI interaction, file variant or OS version
Before/after fixture invariance	Synthetic benchmark contents, name, size, timestamps and xattrs unchanged	Not proof that every filesystem or OS has no side effects
Local UI / build checks	Previous iteration checks covered layout, About, bundle delivery and signing	Not a new exhaustive manual regression or a security audit

Next measurement plan - proposed

Use a consented, representative local corpus. Record cold/warm launch, keypress-to-display p50/p95, peak resident memory, very large folders and mixed formats. Compare the same machine and fixtures before/after. No latency SLO or user satisfaction score is claimed until these measurements exist.

Evidence basis: work/performance/results-2026-10-11.json and the latest saved 53-test run of 11 October 2026. Source and test mapping appears in the companion backlog.

07 / TRUST AND RELEASE READINESS

Read-only is a product promise, not a security sandbox

The current app exposes no photo editing commands and uses read-only paths for images. A source review found no app networking, embedded credentials or third-party package dependencies in the reviewed project. These checks reduce specific concerns; they do not prove absence of vulnerabilities.

Risk / gap	Consequence	Recommended next action
Intentional non-sandboxed process	An exploit would have the process's permitted filesystem access	Correct README permission claims; assess sandboxed folder access separately
Full-detail decode has no hard pixel budget	A huge or hostile image can pressure memory or make viewing unresponsive	Add tested dimension/decoded-byte safeguards before broadly promoting downloads
Native decoder attack surface	Image parsing depends on macOS ImageIO and OS patching	Keep OS current; test malformed inputs; do not claim parser immunity
Local build data in the project tree	Accidental publication can expose paths and include large generated outputs	Stage a clean allowlist; exclude work, DerivedData, logs, personal photos and user settings
Missing explicit local/regular-file guard	Unexpected URL or special-file input may reach synchronous decoding	Validate local, readable regular files and add rejection tests
No selected license / release security policy	Reuse rights and vulnerability reporting are unclear	Approve license and asset provenance; add a reporting route and clean-clone checks

Source release versus downloadable app

Publishing source is separate from distributing a trusted binary. The personal build is ad-hoc signed, without hardened runtime or Developer ID notarisation. Document that status accurately; decide signing, hardened runtime and notarisation before offering a general-audience download.

Review scope: current Swift, project settings, plist/privacy manifest, tests and packaging inventory. No penetration test, fuzzing campaign, CodeQL run or formal audit was performed for these documents. Finding details and release gates are in the backlog.

08 / ROADMAP AND OWNERSHIP

Finish the release story

Next step	Owner / decision	Exit condition
1. Prepare source release	Nikhil approves account, repository name, visibility and license	Clean reviewed file list, truthful README, license/provenance and private reporting route
2. Verify reproducibility	Engineering work with PM acceptance	Fresh-clone build/test on documented Xcode; generated fixtures do not depend on private files
3. Harden for broader use	Prioritise image resource limits and negative tests	Oversized/malformed inputs fail safely; responsiveness and memory bounds measured
4. Publish source	Explicit publication approval required	Public repository, verified contents, build instructions and portfolio documents
5. Consider binary release	Separate trust/distribution choice	Signing/notarisation decision, clean install smoke test and checksums

Open decisions

Repository/account and name; permissive versus reciprocal licensing; artwork provenance; acceptable large-image limits; whether to distribute binaries; support expectations; a security-report contact. None is silently resolved by this PRD.

Portfolio framing

Suggested summary for Nikhil

I led a personal macOS utility from a tightly scoped problem through iterative usability refinement and evidence-led performance work. I used Codex for implementation support, kept ownership of product decisions, and turned feedback into explicit behaviours, tests and release criteria.

Source basis and companion document

Requirements: Nikhil's conversation history. Implementation: PhotoViewerApp.swift, ApplicationState.swift, ContentView.swift, ImageCatalog.swift, ImageLoader.swift, ImageScrollView.swift and ZoomPolicy.swift. Tests: five PhotoViewerTests suites. Performance: synthetic benchmark source and saved results. The delivery backlog translates this contract into epics, features, stories and acceptance criteria.

External release guidance checked on 11 October 2026: GitHub: adding local code; repository licensing; secret scanning; CodeQL scanning. Links are clickable.

No repository was created and no code was uploaded as part of preparing these documents.